UNIVERSIDADE FEDERAL DE OURO PRETO – UFOP
INSTITUTO DE CIÊNCIAS EXATAS E BIOLÓGICAS – ICEB
DEPARTAMENTO DE COMPUTAÇÃO – DECOM

BCC203 – ESTRUTURA DE DADOS II

RELATÓRIO PRÁTICO I:

ANÁLISE COMPARATIVA DE ALGORITMOS DE PESQUISA EM MEMÓRIA SECUNDÁRIA

Acesso Sequencial Indexado, Árvore Binária de Pesquisa Externa, Árvore B e Árvore B*

Autores:

Kaio Alves Simões

Luiz Henrique Silva Sampaio

Victor Kangussu

Vinícius Brandão

Professor:

Prof. Guilherme Tavares de Assis

Ouro Preto – MG
2026

Sumário

1 Introdução **1**

2 Ferramentas Utilizadas e Ambiente Experimental **2**
2.1 Linguagem, Compilador e Ferramentas de Construção 2
2.2 Especificações do Ambiente de Execução (Hardware e Software) 2
2.3 Metodologia Experimental e Métricas Coletadas 3

3 Algoritmos de Pesquisa **5**
3.1 Acesso Sequencial Indexado (Indexed Sequential Access – ISA) 5
3.1.1 Princípio de Funcionamento e Implementação 5
3.1.2 Resultados Experimentais e Tabela de Benchmark 6
3.2 Árvore Binária de Pesquisa Externa (Binary Tree) 7
3.2.1 Princípio de Funcionamento e Implementação 7
3.2.2 Resultados Experimentais e Tabela de Benchmark 8
3.3 Árvore B (B-Tree) 10
3.3.1 Princípio de Funcionamento e Implementação 10
3.3.2 Resultados Experimentais e Tabela de Benchmark 11
3.4 Árvore B* (B*-Tree) 12
3.4.1 Princípio de Funcionamento e Implementação 12
3.4.2 Resultados Experimentais e Tabela de Benchmark 14

4 Comparação Geral e Discussão dos Algoritmos **15**
4.1 Custo e Desempenho no Pré-processamento 15
4.2 Eficiência na Fase de Pesquisa 16
4.3 Sensibilidade ao Ordenamento Inicial da Entrada 17
4.4 Impacto do Subsistema de Memória e Caches (Cold vs. Warm) 17
4.5 Síntese Comparativa Geral 17

5 Conclusão **18**

1 Introdução

O processamento e a recuperação de grandes volumes de informações constituem um dos pilares centrais da Ciência da Computação contemporânea. Em cenários reais, bancos de dados corporativos e sistemas de arquivos frequentemente manipulam volumes de registros que excedem substancialmente a capacidade da memória principal (RAM) disponível, ou demandam estruturas de indexação persistentes entre execuções. Nesses contextos, as estruturas de dados clássicas em memória volátil tornam-se inadequadas devido ao custo de acesso à memória secundária (discos magnéticos ou unidades de estado sólido – SSDs), cuja latência é ordens de magnitude superior à da memória primária.

Na disciplina de **Estrutura de Dados II (BCC203)**, o estudo de métodos de pesquisa externa visa projetar e avaliar algoritmos capazes de minimizar a quantidade de transferências de blocos (páginas) entre o disco e a RAM, otimizando o tempo de resposta e o consumo de I/O (*Input/Output*).

O presente trabalho prático analisa, implementa e compara experimentalmente quatro estruturas de dados e algoritmos de pesquisa externa, contemplando diferentes modelos de residência de memória para seus índices:

1. **Acesso Sequencial Indexado (ISA – *Indexed Sequential Access*):** implementado com tabela de índices em memória secundária (arquivo de cache);
2. **Árvore Binária de Pesquisa Externa:** implementada integralmente em memória secundária, com nós persistidos em arquivo binário;
3. **Árvore B (Ordem M):** implementada em **memória secundária**, serializando seus nós multi-vias diretamente em arquivo binário persistente (`.btree`), de modo que a navegação entre níveis da árvore demanda transferências de blocos do disco para a RAM;
4. **Árvore B* (com separação entre nós de índice e folhas de dados):** implementada em **memória principal**, mantendo sua hierarquia de nós dinamicamente em memória RAM e acessando a memória secundária estritamente para carregar a página do registro final resgatado.

Modelo de Registros e Paginação: Os experimentos manipulam registros representados pela estrutura `Item`, onde cada elemento é constituído por:

- Uma chave primária inteira de 32 bits (`int key`);
- Um valor numérico de 64 bits (`long int value`);
- Um arranjo fixo de caracteres com 5.000 bytes (`std::array<char, 5000> text`).

Dada a dimensão volumétrica de cada item (aproximadamente 5.012 bytes), a transferência entre memória secundária e primária ocorre em blocos fixos denominados *páginas*, onde cada página encapsula exatamente PAGE_SIZE = 100 registros.

Delimitação de Escopo: Ressalta-se expressamente que os utilitários de geração sintética de arquivos binários (*gerador de arquivos*) foram concebidos com a finalidade exclusiva de prover massas de dados de teste padronizadas para a bancada experimental. Conforme as especificações deste relatório, o gerador de arquivos é completamente omitido do escopo analítico, concentrando-se todo o estudo nas fases de pré-processamento/construção dos índices, nos algoritmos de pesquisa e nas métricas empíricas de desempenho.

2 Ferramentas Utilizadas e Ambiente Experimental

Para assegurar reprodutibilidade científica, robustez de engenharia de software e precisão nas coletas cronométricas, todo o ecossistema experimental foi cuidadosamente padronizado.

2.1 Linguagem, Compilador e Ferramentas de Construção

O projeto foi integralmente desenvolvido na linguagem **C++23**, tirando proveito de construções semânticas modernas da biblioteca padrão, como abstrações de sistemas de arquivos (`std::filesystem`), semântica de valores opcionais (`std::optional`), medição de alta precisão temporal (`std::chrono`) e encapsulamento modular:

- **Compilador:** Clang++ 21.1 / GCC 15.3, operando sob o padrão ISO C++23 (`-std=c++23`);
- **Sinalizadores de Compilação:** Verificação estrita de conformidade (`-Wall`, `-Wextra`, `-Wpedantic`) e compilação em perfil Release com otimização ativada (`-O3`);
- **Automação de Build:** CMake (versão 3.20+) gerenciando alvos de compilação, dependências e geração da base de compilação (`compile_commands.json`);
- **Ambiente Reproduzível:** Nix Flakes e *devenv*, garantindo paridade determinística de ferramentas de compilação, bibliotecas de sistema e pacotes LaTeX;
- **Qualidade de Código:** `clang-format` (padronização de estilo) e `clang-tidy` (linter e análise estática).

2.2 Especificações do Ambiente de Execução (Hardware e Software)

Os testes de desempenho foram executados no ambiente com as seguintes especificações físicas e lógicas:

Tabela 1: Especificação técnica da estação de testes.

Componente	**Especificação Técnica**
Processador (CPU)	*[Ex: AMD Ryzen 7 5800X / Intel Core i7-12700H]*
Arquitetura e Núcleos	*[Ex: x86_64, 8 núcleos / 16 threads, Clock base 3.4 GHz]*
Memória RAM	*[Ex: 32 GB DDR4 3200 MHz]*
Armazenamento Secundário	*[Ex: NVMe PCIe 4.0 SSD M.2 1TB / Leitura sequencial 5000 MB/s]*
Sistema Operacional	Linux *[Ex: NixOS 24.11 / Ubuntu 24.04 LTS]*
Versão do Kernel	Linux Kernel *[Ex: 6.6.x-generic]*
Sistema de Arquivos	*[Ex: ext4 / btrfs / zfs]*

2.3 Metodologia Experimental e Métricas Coletadas

A avaliação de cada algoritmo é realizada em duas fases distintas, conforme implementado no módulo central de instrumentação:

1. **Fase de Pré-processamento:** Compreende a varredura inicial do arquivo de dados e a geração ou carga da estrutura de índice correspondente (construção da tabela de índice ou árvore em memória secundária/primária);

2. **Fase de Pesquisa:** Compreende a localização da chave desejada na estrutura indexada, o mapeamento do número da página no arquivo de dados, a transferência da página de 100 itens para a RAM e a recuperação do registro completo.

Para cada fase, as seguintes grandezas são registradas através do coletor estático `Metrics`:

- **Tempo de Execução (T, em milissegundos):** Mensurado em alta resolução via `std::chrono::high_resolution_clock`;

- **Transferências de Disco para RAM (R):** Contagem exata de requisições de blocos/páginas transferidos da memória secundária para a memória principal;

- **Comparações de Chaves (C):** Contagem estrita de avaliações relacionais efetuadas entre a chave buscada e as chaves dos registros indexados.

Regimes de Execução – Cold vs. Warm e Modelos de Memória: O comportamento dos métodos de pesquisa externa é analisado sob dois regimes operacionais distintos em relação ao estado de persistência da estrutura de índice:

- **Cold (Execução a Frio / Cold Start):** Para as estruturas baseadas em memória secundária (ISA, Árvore Binária e Árvore B), ocorre quando **é preciso criar o arquivo de cache/índice em disco** correspondente. Nesse cenário, o arquivo de índice não existe previamente em disco, exigindo que a fase de pré-processamento realize a varredura completa do arquivo de dados e construa/persista a estrutura de índice do zero. Para a Árvore B* em memória principal, corresponde à carga e montagem completa da árvore na memória RAM a partir da leitura sequencial do arquivo de dados;

- **Warm (Execução a Quente / Warm Start):** Para as estruturas baseadas em memória secundária, ocorre quando **o arquivo de cache/índice já existe** no disco previamente e está válido. O algoritmo detecta o arquivo persistente pré-existente e o reaproveita/carrega diretamente, eliminando a sobrecarga de reconstrução do índice a partir do arquivo de dados. Para a Árvore B*, corresponde à reutilização da estrutura de dados já instanciada em memória principal.

Metodologia de Coleta de Dados e Formação das Tabelas: Para garantir rigor científico, reprodutibilidade e mitigar ruídos estocásticos decorrentes de interrupções de hardware, paginação do sistema operacional ou flutuações de I/O em disco, cada cenário de teste experimental deve ser executado **100 vezes sob as mesmas condições**. Todos os dados mensurados nessas execuções são considerados **diretamente para o benchmark**, eliminando-se qualquer agregação ou cálculo de mediana intermediária, e reportando-se as métricas sob a ótica dos percentis **p50 (mediana)**, **p75** e **p99**.

Para cada configuração avaliada (combinação de algoritmo, volume de dados N, estado de ordenação e regime operacional), adota-se estritamente o seguinte procedimento metodológico:

1. **Execuções Sob as Mesmas Condições (100 Repetições):** Cada teste é executado exatamente **100 vezes sob idênticas condições operacionais e estruturais**, coletando-se diretamente as métricas de tempo de execução, transferências de disco e comparações de chaves para compor o conjunto amostral do benchmark sem agregações prévias;

2. **Checagem de Presença da Chave e Regeneração de Entrada:** Mantém-se a verificação mandatória de que a chave pesquisada deve estar **estritamente contida no arquivo de entrada**, considerando-se válidos exclusivamente os testes realizados com chaves presentes nos dados. Caso o teste falhe por falta da chave na entrada gerada, a execução é sumariamente desconsiderada e o teste **deve ser refeito com uma nova entrada**, reiterando-se o ciclo até totalizar as 100 execuções válidas com a chave presente;

3. **Controle de Regime (Cold Start vs. Warm Start):**

   - No regime **Cold Start**, assegura-se expressamente que o arquivo de índice/cache das estruturas em memória secundária **não exista no disco** antes do início da execução (removendo qualquer arquivo persistente prévio correspondente), forçando o algoritmo a processar o arquivo de entrada do zero e persistir a estrutura de índice em disco (ou instanciá-la em RAM no caso da Árvore B*);
   - No regime **Warm Start**, a execução é realizada preservando-se a estrutura previamente construída e validada (reaproveitando o arquivo persistido em disco para ISA, Árvore Binária e Árvore B, ou a árvore já montada em memória principal para a Árvore B*), avaliando o custo de busca com o índice pré-existente sob as mesmas condições.

4. **Consolidação Direta no Benchmark (p50, p75 e p99):** Os dados das 100 execuções válidas são computados diretamente para a formação das estatísticas de benchmark:

   - **Tempos de Execução:** Apuram-se os percentis da distribuição direta de tempos das 100 amostras: o **p50 (Mediana)** expressa a tendência central e o comportamento típico do sistema; o **p75 (75º Percentil)** reflete o patamar superior de tempo na maior parte dos ensaios; e o **p99 (99º Percentil)** captura a latência de cauda (*tail latency*) e eventuais gargalos transitórios de I/O em disco;
   - **Transferências e Comparações:** Dada a natureza algorítmica uniforme dessas grandezas para uma mesma topologia de busca, reporta-se o valor mediano central (**p50**) obtido diretamente das 100 execuções.

Cenários de Entrada (Situações): Os ensaios foram conduzidos considerando três estados estruturais dos dados originais:

- **Situação 1 – Arquivo Ordenado Ascendentemente;**

- **Situação 2 – Arquivo Ordenado Descendentemente;**
- **Situação 3 – Arquivo Desordenado (Aleatório/Embaralhado).**

Para cada cenário, foram submetidos volumes variados de registros, tipicamente parametrizados em ordens de grandeza incrementais: $N \in \{20, 2.000, 20.000, 200.000, 2.000.000\}$.

3 Algoritmos de Pesquisa

Nesta seção, cada um dos quatro métodos de pesquisa externa implementados é descrito detalhadamente em termos de seu modelo teórico, arquitetura de implementação em C++, funcionamento das fases de pré-processamento e busca, acompanhado de sua respectiva tabela comparativa de benchmarks e análise de comportamento.

3.1 Acesso Sequencial Indexado (Indexed Sequential Access – ISA)

3.1.1 Princípio de Funcionamento e Implementação

O método de **Acesso Sequencial Indexado** é uma das abordagens pioneiras para redução de acessos em arquivos externos ordenados. A técnica baseia-se na criação de uma tabela de índices esparsa, na qual cada entrada mapeia a menor chave contida em uma página de dados ao respectivo índice da página no arquivo binário.

Arquitetura e Classes: A implementação foi modularizada no *namespace* `Algorithm::IndexedSequentialAccess`:

- `Cache`: Classe responsável pela criação, validação temporal e manipulação do arquivo de índice (`.cache`). Cada entrada (`Cache::Entry`) armazena um par `(key, pageIndex)`. A estrutura conta ainda com metadados (`lastModification` e `size`) para garantir que o cache em disco seja invalidado caso o arquivo de dados sofra alterações;
- `ISA`: Classe de alto nível que orquestra a consulta. Recebe a instância compartilhada de `File` e coordena a consulta ao índice e a posterior recuperação da página de dados.

Fase de Pré-processamento: Durante a construção do índice (`Cache::BuildCache`), o arquivo de dados original é percorrido sequencialmente de página em página. Para cada página, a menor chave (o primeiro item da página) é extraída juntamente com seu número de página e gravada no arquivo de índice. Como o índice armazena apenas uma entrada por página, o tamanho do arquivo de índice é $\approx \frac{1}{\text{PAGE_SIZE}}$ do tamanho do arquivo de dados original, permitindo carregamento rápido ou busca direta.

Fase de Pesquisa: Ao consultar uma chave K:

1. Uma busca binária é executada diretamente nas entradas do arquivo de índice (`Cache::Search(key)`), localizando a página candidata onde K pode residir (isto é, a página cujo menor item satisfaz chave_pagina $\leq K <$ chave_proxima_pagina);

2. Se nenhuma página compatível for encontrada, a busca é finalizada com ausência;

3. Caso contrário, apenas a página indicada é carregada do disco para a memória principal via `File::GetPageAt(pageIndex)`;

4. Realiza-se uma busca sequencial linear sobre os itens válidos da página em RAM, comparando a chave procurada até localizá-la ou esgotar a página.

Restrições Estruturais: Por depender da pressuposição de monotonicidade entre as páginas e dentro de cada página para delimitar os intervalos de busca, o método exige obrigatoriamente que o arquivo esteja **ordenado ascendentemente** (Situação 1). A tentativa de execução sobre arquivos descendentemente ordenados (Situação 2) ou desordenados (Situação 3) é rejeitada pelo sistema por inconsistência teórica do índice esparso simples.

3.1.2 Resultados Experimentais e Tabela de Benchmark

A Tabela 2 consolida as medições obtidas pelo método ISA em diferentes escalas volumétricas, contrastando execuções *Cold* (criação mandatória do arquivo de cache) e *Warm* (reaproveitamento do arquivo de cache existente). Conforme detalhado na metodologia, os tempos de execução reportam os percentis p50, p75 e p99 obtidos diretamente a partir das 100 execuções sob as mesmas condições (com a chave garantidamente presente na entrada), enquanto transferências de disco e comparações de chaves expressam a mediana central apurada (p50).

Tabela 2: Benchmark experimental – Acesso Sequencial Indexado (ISA).

			Pré-processamento					**Pesquisa**				
			Tempo (ms)			**Leituras**	**Comparações**	**Tempo (ms)**			**Leituras**	**Comparações**
Situação	**Regime**	**Qtd (N)**	**p50**	**p75**	**p99**	**(p50)**	**(p50)**	**p50**	**p75**	**p99**	**(p50)**	**(p50)**
Ascendente	Cold	20	0.35	0.37	0.46	1	0	0.04	0.04	0.09	2	10
Ascendente	Cold	2.000	1.18	1.24	1.50	20	0	0.06	0.07	0.09	6	48
Ascendente	Cold	20.000	8.63	9.02	10.56	200	0	0.07	0.08	0.10	9	58
Ascendente	Cold	200.000	84.19	87.05	110.95	2.000	0	0.08	0.09	0.10	12	57
Ascendente	Cold	2.000.000	17936.20	17951.80	18003.01	20.000	0	0.09	0.32	1.30	15	72
Ascendente	Warm	20	0.02	0.02	0.04	1	0	0.28	0.30	0.42	2	12
Ascendente	Warm	2.000	0.02	0.02	0.05	1	0	0.32	0.34	0.38	5	54
Ascendente	Warm	20.000	0.02	0.02	0.05	1	0	0.32	0.34	0.44	9	56
Ascendente	Warm	200.000	0.02	0.02	0.05	1	0	0.33	0.35	0.39	12	57
Ascendente	Warm	2.000.000	0.02	0.02	0.03	1	0	0.34	0.36	1.15	15	68
Descendente	Cold/Warm	Todos	*Não suportado (operação rejeitada por violação de ordenação)*									
Desordenado	Cold/Warm	Todos	*Não suportado (operação rejeitada por violação de ordenação)*									

Discussão dos Resultados: Os resultados experimentais sintetizados na Tabela 2 comprovam a eficiência do Acesso Sequencial Indexado (ISA) na contenção de operações de E/S, ao mesmo tempo em que delimitam sua dependência estrita das condições da entrada:

Na fase de pré-processamento sob regime *Cold Start*, o custo de construção do arquivo de índice (`.cache`) apresenta comportamento estritamente linear $O(N)$, efetuando exatamente uma leitura por página ($P = \lceil N/\text{PAGE_SIZE} \rceil$) e zero comparações de chaves, demandando desde 0,35 ms para $N = 20$ até aproximadamente 17,9 s para 2 milhões de registros. No regime *Warm Start*, a detecção do índice persistente pré-existente e a validação de metadados reduzem o pré-processamento a uma única leitura ($R = 1$) e

tempos medianos de apenas 0,02 ms para qualquer volume, demonstrando a amortização completa do índice em execuções subsequentes.

Na fase de pesquisa, o método destaca-se pela expressiva parcimônia de I/O, demandando apenas $R \approx \lceil \log_2(N/\text{PAGE_SIZE}) \rceil + 1$ transferências de disco (somando a busca binária no arquivo de índice à carga da página de dados). Mesmo diante de $N = 2.000.000$ (20.000 páginas), foram necessárias somente 15 leituras de disco, mantendo o tempo mediano de consulta abaixo de 0,35 ms. Além disso, como o índice esparso armazena apenas a menor chave de cada página e o respectivo índice de bloco, a tabela ocupa cerca de 1% do tamanho do arquivo original, economizando espaço em disco e largura de banda ao evitar a duplicação dos registros volumosos (5.000 bytes por item).

Ressalta-se, contudo, a principal restrição estrutural do algoritmo: seu funcionamento é **estritamente restrito ao caso em que o arquivo de entrada se encontra ordenado ascendentemente** (Situação 1). A corretude da busca binária fundamenta-se na premissa de monotonicidade das chaves para delimitar os intervalos de cada página ($\text{chave_pagina}_i \leq K < \text{chave_pagina}_{i+1}$). Em arquivos ordenados descendentemente (Situação 2) ou desordenados (Situação 3), essa propriedade é completamente violada, inviabilizando a delimitação por busca binária e gerando falhas ou falsos negativos imediatos. Por esse motivo, tais cenários são sumariamente rejeitados pela implementação. Sem a garantia de ordenação prévia dos registros de entrada, o método torna-se inoperante, demandando uma etapa prévia e onerosa de ordenação externa sobre o arquivo de dados.

3.2 Árvore Binária de Pesquisa Externa (Binary Tree)

3.2.1 Princípio de Funcionamento e Implementação

A **Árvore Binária de Pesquisa Externa** adapta o conceito fundamental de árvores binárias de busca (BST) para o armazenamento permanente em disco. Diferentemente de uma árvore binária em memória RAM em que os apontadores são endereços de memória virtual (*pointers*), nesta implementação os nós residem sequencialmente em um arquivo binário e os apontadores para subárvores esquerda e direita são índices numéricos de nós (offsets baseados em registro).

Arquitetura e Classes: Implementada no *namespace* `Algorithm::BinaryTree`:

- `BTreeFile::Node`: Estrutura de nó compacta contendo a chave inteira (`int key`), o índice da página correspondente no arquivo de dados (`uint64_t pageIndex`) e dois descritores inteiros de posição para os nós filhos (`uint64_t left` e `uint64_t right`);
- `BTreeFile`: Classe encarregada da criação e da leitura nó a nó via acesso randômico no arquivo binário de índice da árvore;
- `BTree`: Fachada que conecta o índice da árvore binária ao arquivo de dados original.

Fase de Pré-processamento: No método `BTreeFile::BuildFile`, o arquivo de dados de entrada é percorrido bloco a bloco. Cada registro é submetido à rotina de inserção na árvore binária:

1. O primeiro item inserido torna-se o nó raiz (posição 0 do arquivo de árvore);

2. Para cada novo item, a inserção percorre o arquivo de nós a partir da raiz, comparando as chaves. Leituras sucessivas de nós em disco são executadas para navegar até um apontador nulo (`left == 0` ou `right == 0`);

3. Ao atingir a folha correspondente, o novo nó é anexado ao final do arquivo de índice e o apontador do nó pai é devidamente atualizado.

Fase de Pesquisa: Dada a chave procurada K:

1. A busca inicia-se na raiz (`Search(key)`). O nó raiz é lido do disco;

2. Se nó.key == K, a busca no índice encerra com sucesso, recuperando o `pageIndex`;

3. Se $K <$ nó.key, a busca avança para o nó apontado por `nó.left`; caso contrário, para `nó.right`;

4. Se um apontador nulo for atingido, a chave não existe;

5. Após localizar o nó, a página correspondente no arquivo de dados é carregada para a RAM (`File::GetPageAt`) e submetida a varredura linear para recuperar o `Item` integral.

Sensibilidade à Ordenação e Degeneração: Por se tratar de uma árvore binária sem rebalanceamento automático, o algoritmo exibe extrema sensibilidade à ordem dos dados de entrada:

- **Arquivo Ordenado (Ascendente ou Descendente):** A estrutura degenera completamente em uma lista linear encadeada de altura $h = N$. Cada inserção e cada busca passa a demandar $O(N)$ leituras de disco, provocando volumoso tráfego de I/O e degradação severa de desempenho;

- **Arquivo Desordenado (Aleatório):** A estrutura tende a um comportamento balanceado médio de altura $h \approx O(\log_2 N)$, reduzindo consideravelmente as leituras.

3.2.2 Resultados Experimentais e Tabela de Benchmark

A Tabela 3 sintetiza os resultados de benchmark da Árvore Binária Externa abrangendo os três tipos de entrada e regimes *Cold* e *Warm*. Os tempos de execução reportam os percentis p50, p75 e p99 obtidos diretamente a partir das 100 execuções sob as mesmas condições (com a chave garantidamente presente na entrada), além das medianas consolidadas de transferências e comparações (p50).

Tabela 3: Benchmark experimental – Árvore Binária de Pesquisa Externa.

			Pré-processamento					Pesquisa				
			Tempo (ms)			Leituras	Comparações	Tempo (ms)			Leituras	Comparações
Situação	**Regime**	**Qtd (N)**	**p50**	**p75**	**p99**	**(p50)**	**(p50)**	**p50**	**p75**	**p99**	**(p50)**	**(p50)**
Ascendente	Cold	20	0.50	0.51	0.56	103	201	0.03	0.04	0.06	7	21
Ascendente	Cold	2.000	1383.97	1388.12	1400.48	1.000.022	2.000.001	0.84	1.20	1.57	486	1.033
Ascendente	Cold	20.000	-	-	-	-	-	-	-	-	-	-
Ascendente	Cold	200.000	-	-	-	-	-	-	-	-	-	-
Ascendente	Cold	2.000.000	-	-	-	-	-	-	-	-	-	-
Ascendente	Warm	20	0.02	0.02	0.02	1	0	0.22	0.23	0.27	7	22
Ascendente	Warm	2.000	0.02	0.02	0.04	1	0	0.93	1.35	1.77	444	926
Ascendente	Warm	20.000	-	-	-	-	-	-	-	-	-	-
Ascendente	Warm	200.000	-	-	-	-	-	-	-	-	-	-
Ascendente	Warm	2.000.000	-	-	-	-	-	-	-	-	-	-
Descendente	Cold	20	0.49	0.50	0.57	103	201	0.03	0.03	0.05	7	21
Descendente	Cold	2.000	1390.51	1393.18	1415.33	1.000.022	2.000.001	0.87	1.28	1.53	529	1.088
Descendente	Cold	20.000	-	-	-	-	-	-	-	-	-	-
Descendente	Cold	200.000	-	-	-	-	-	-	-	-	-	-
Descendente	Cold	2.000.000	-	-	-	-	-	-	-	-	-	-
Descendente	Warm	20	0.02	0.02	0.02	1	0	0.22	0.23	0.28	7	24
Descendente	Warm	2.000	0.02	0.02	0.03	1	0	1.05	1.41	1.76	530	1.086
Descendente	Warm	20.000	-	-	-	-	-	-	-	-	-	-
Descendente	Warm	200.000	-	-	-	-	-	-	-	-	-	-
Descendente	Warm	2.000.000	-	-	-	-	-	-	-	-	-	-
Desordenado	Cold	20	0.44	0.44	0.51	67	129	0.03	0.03	0.03	6	20
Desordenado	Cold	2.000	142.27	151.80	153.84	104.687	209.331	0.08	0.15	0.49	18	101
Desordenado	Cold	20.000	1257.67	1260.67	1271.47	836.366	1.672.329	0.09	0.13	0.39	24	117
Desordenado	Cold	200.000	9295.27	9312.37	9379.65	5.857.329	11.710.655	0.09	0.10	0.23	25	104
Desordenado	Cold	2.000.000	-	-	-	-	-	-	-	-	-	-
Desordenado	Warm	20	0.02	0.02	0.04	1	0	0.22	0.23	0.29	5	18
Desordenado	Warm	2.000	0.02	0.02	0.04	1	0	0.29	0.36	0.76	26	118
Desordenado	Warm	20.000	0.02	0.02	0.03	1	0	0.29	0.32	0.57	26	113
Desordenado	Warm	200.000	0.02	0.02	0.04	1	0	0.29	0.30	0.38	26	106
Desordenado	Warm	2.000.000	-	-	-	-	-	-	-	-	-	-

Discussão dos Resultados: Os resultados apresentados na Tabela 3 evidenciam o impacto direto da ausência de um sistema de balanceamento na Árvore Binária de Pesquisa Externa. Nas Situações 1 (Ascendente) e 2 (Descendente), a inserção sequencial das chaves direciona todos os nós para um único ramo, degenerando a estrutura em uma lista encadeada linear de altura $h = O(N)$. Com isso, a inserção de cada novo elemento exige percorrer e ler sequencialmente todos os nós anteriores no arquivo (`.binarytree`), elevando o custo de construção no regime *Cold Start* à complexidade quadrática $O(N^2)$ em tempo, comparações e leituras de disco. Esse comportamento é comprovado experimentalmente para $N = 2.000$, onde foram necessárias 1.000.022 leituras e 2.000.001 comparações (tempo mediano de $\approx 1{,}39$ s). Essa explosão de operações de I/O inviabiliza as entradas maiores ($N \in \{20.000, 200.000, 2.000.000\}$), fazendo com que a execução ultrapasse o tempo limite e sofra invariavelmente de *timeout*. Na fase de pesquisa, a degeneração também impõe um custo linear $O(N)$, demandando cerca de 500 leituras de disco para localizar uma chave em $N = 2.000$.

Em contrapartida, na Situação 3 (Desordenado), a distribuição aleatória das chaves preserva uma altura média proporcional a $O(\log_2 N)$, viabilizando a construção até $N = 200.000$ e mantendo a pesquisa eficiente, com apenas 18 a 25 leituras de disco e latência inferior a 0,10 ms a frio. No entanto, para $N = 2.000.000$, a sobrecarga de I/O em acessos aleatórios disco a disco também culmina em *timeout*. Por fim, no regime *Warm Start*, a reutilização do índice persistente pré-existente reduz o pré-processamento a uma única leitura de validação de metadados ($R = 1$) e apenas 0,02 ms, demonstrando a plena amortização do custo de construção nos cenários em que o arquivo foi previamente gerado.

3.3 Árvore B (B-Tree)

3.3.1 Princípio de Funcionamento e Implementação

A **Árvore B** é uma estrutura de pesquisa balanceada multi-vias concebida especificamente para operar com eficiência sobre dispositivos de memória secundária. No âmbito deste projeto, a Árvore B foi **efetivamente implementada em memória secundária** através da classe `BTreeFile`, persistindo todos os seus nós e metadados diretamente em um arquivo binário em disco (com extensão `.btree`).

Em uma Árvore B de ordem M (onde adotou-se o parâmetro alinhado a PAGE_SIZE = 100), cada nó alocado em disco armazena múltiplos pares de chave/dado e múltiplos apontadores para nós filhos, propiciando um fator de ramificação (*fan-out*) expressivamente elevado que mantém a altura da árvore tipicamente restrita a ≤ 3 níveis mesmo diante de milhões de registros.

Arquitetura e Implementação em Memória Secundária: A estrutura e a persistência em disco foram modeladas no *namespace* `Algorithm::BTree`:

- `IBTreeData::Entry`: Entrada individual contendo a chave inteira (`int key`) e a referência ordinal da página no arquivo de dados (`uint64_t pageIndex`);
- `IBTreeData::Node`: Bloco de nó serializável de tamanho fixo em disco contendo até PAGE_SIZE entradas ordenadas internamente por chave e até PAGE_SIZE + 1 apontadores (`int64_t nodePos`) para os nós filhos, além dos controles de ocupação (`size`) e identificação de folha (`isLeaf()`). Os apontadores de filhos não são endereços de memória virtual, mas sim posições ordinais de nós gravados no arquivo de índice;
- `BTreeFile`: Implementação da Árvore B em **memória secundária**. Gerencia o fluxo binário (`std::fstream`), organiza o cabeçalho de metadados (`Metadata`: data de modificação da entrada, total de itens e offset do nó raiz `rootIndex`) e executa o endereçamento posicional dos nós via cálculo direto de deslocamento em bytes:

$$\text{Offset}(\text{nodeIndex}) = \text{sizeof}(\text{Metadata}) + \text{nodeIndex} \times \text{sizeof}(\text{Node}) \tag{1}$$

- `BTree`: Fachada de alto nível que orquestra as chamadas ao índice persistido em memória secundária e coordena a subsequente leitura do registro na página correspondente do arquivo de dados.

Fase de Pré-processamento e Persistência em Disco: A construção do índice na Árvore B em memória secundária reflete o custo real de I/O em disco:

1. No regime **Cold Start**, o arquivo binário `.btree` é criado em disco. O arquivo de entrada é percorrido página a página, e cada registro é submetido à rotina de inserção persistente;
2. A inserção desce recursivamente pelo arquivo de índice lendo os nós em disco (`ReadNode`). Ao atingir a folha, insere a entrada ordenadamente;
3. Em caso de transbordamento (ultrapassando PAGE_SIZE entradas), realiza-se a divisão do nó (*split*), alocando um novo nó irmão no final do arquivo (`AppendNode`), promovendo a chave mediana e regravando os blocos alterados diretamente em disco (`WriteNode`);

4. Se a raiz transbordar, uma nova raiz é anexada ao arquivo de índice e o cabeçalho de metadados é atualizado com o novo `rootIndex`;

5. No regime **Warm Start**, o arquivo `.btree` pré-existente no disco tem seus metadados validados contra o arquivo de dados original (`ValidateFile`), permitindo reaproveitamento instantâneo sem necessidade de reconstrução.

Fase de Pesquisa em Memória Secundária: A consulta por uma chave K em memória secundária opera nó a nó sob demanda:

1. A leitura inicia no nó raiz recuperado do arquivo `.btree` em disco via `ReadNode(file, rootIndex, node)`;

2. Realiza-se a busca pela chave dentro do nó carregado em RAM;

3. Se uma entrada possuir chave idêntica a K, a busca no índice encerra com sucesso, recuperando o `pageIndex`;

4. Se a chave K estiver delimitada por entradas adjacentes, a busca descende para o nó filho correspondente em nodePos, emitindo uma nova leitura de disco (`ReadNode`) para transferir o nó filho do arquivo para a RAM;

5. O processo itera até localizar a chave ou alcançar uma folha sem sucesso;

6. Ao término da busca no índice com sucesso, a página de dados apontada por `pageIndex` é lida da memória secundária via `File::GetPageAt(pageIndex)`, totalizando $h + 1$ transferências de disco (onde h é o número de nós lidos no arquivo de índice).

3.3.2 Resultados Experimentais e Tabela de Benchmark

A Tabela 4 reporta os dados cronométricos e de I/O coletados nos testes com a Árvore B nos regimes *Cold* e *Warm*. Os tempos de execução reportam os percentis p50, p75 e p99 obtidos diretamente a partir das 100 execuções sob as mesmas condições (com a chave garantidamente presente na entrada), além das medianas consolidadas de transferências e comparações (p50).

Tabela 4: Benchmark experimental – Árvore B.

			Pré-processamento					Pesquisa				
			Tempo (ms)			Leituras	Comparações	Tempo (ms)			Leituras	Comparações
Situação	**Regime**	**Qtd (N)**	**p50**	**p75**	**p99**	**(p50)**	**(p50)**	**p50**	**p75**	**p99**	**(p50)**	**(p50)**
Ascendente	Cold	20	0.22	0.25	0.34	20	73	0.03	0.03	0.05	2	23
Ascendente	Cold	2.000	4.46	4.51	6.13	3.918	20.266	0.04	0.05	0.08	3	84
Ascendente	Cold	20.000	55.34	57.40	61.97	54.897	269.193	0.07	0.07	0.09	4	100
Ascendente	Cold	200.000	557.74	582.13	666.14	596.697	3.369.047	0.08	0.09	0.10	4	96
Ascendente	Cold	2.000.000	21117.75	21145.60	32391.72	7.749.396	40.380.512	0.09	0.91	1.58	5	104
Ascendente	Warm	20	0.01	0.01	0.03	1	0	0.14	0.16	0.22	2	27
Ascendente	Warm	2.000	0.01	0.01	0.02	1	0	0.18	0.19	0.24	3	81
Ascendente	Warm	20.000	0.01	0.01	0.02	1	0	0.19	0.20	0.24	4	92
Ascendente	Warm	200.000	0.01	0.02	0.03	1	0	0.19	0.21	0.26	4	94
Ascendente	Warm	2.000.000	0.01	0.02	0.03	1	0	0.21	0.99	2.18	5	102
Descendente	Cold	20	0.20	0.22	0.27	20	278	0.02	0.02	0.03	2	27
Descendente	Cold	2.000	4.15	4.27	4.94	3.918	173.672	0.04	0.04	0.07	3	87
Descendente	Cold	20.000	54.65	54.96	65.22	54.897	1.875.804	0.07	0.08	0.10	4	97
Descendente	Cold	200.000	584.57	625.05	679.13	596.697	19.622.061	0.09	0.09	0.11	4	102
Descendente	Cold	2.000.000	34894.60	34989.62	35046.16	7.749.396	206.263.953	0.09	0.68	1.84	5	106
Descendente	Warm	20	0.01	0.02	0.03	1	0	0.14	0.16	0.21	2	26
Descendente	Warm	2.000	0.01	0.01	0.02	1	0	0.18	0.19	0.25	3	90
Descendente	Warm	20.000	0.02	0.02	0.03	1	0	0.21	0.22	0.33	4	104
Descendente	Warm	200.000	0.02	0.02	0.03	1	0	0.22	0.23	0.32	4	96
Descendente	Warm	2.000.000	0.01	0.02	0.02	1	0	0.20	0.87	1.19	5	96
Desordenado	Cold	20	0.20	0.21	0.30	20	170	0.02	0.02	0.05	2	24
Desordenado	Cold	2.000	4.36	4.37	5.07	3.918	80.521	0.05	0.05	0.07	3	99
Desordenado	Cold	20.000	57.69	59.86	70.69	53.883	898.490	0.07	0.08	0.11	4	100
Desordenado	Cold	200.000	662.09	678.19	742.92	595.683	10.431.817	0.09	0.09	0.11	4	95
Desordenado	Cold	2.000.000	8644.26	8856.06	9512.31	7.599.702	115.703.572	0.09	0.10	0.11	5	109
Desordenado	Warm	20	0.01	0.02	0.02	1	0	0.15	0.16	0.20	2	26
Desordenado	Warm	2.000	0.01	0.01	0.02	1	0	0.19	0.20	0.25	3	102
Desordenado	Warm	20.000	0.01	0.02	0.02	1	0	0.20	0.21	0.29	4	99
Desordenado	Warm	200.000	0.01	0.02	0.03	1	0	0.20	0.21	0.29	4	110
Desordenado	Warm	2.000.000	0.02	0.02	0.02	1	0	0.20	0.21	0.29	5	114

Discussão dos Resultados: *[Espaço para análise do comportamento observado: destacar a imunidade da Árvore B contra variações de ordenação dos arquivos de entrada, a profundidade extremamente baixa (tipicamente ≤ 3 níveis para até 2 milhões de itens com ordem 100) e a estabilidade das leituras de disco requeridas por consulta. Ressalta-se que, devido à sua implementação em memória secundária, cada consulta exige transferências físicas de blocos para cada nó de índice percorrido no arquivo* `.btree` *em disco somadas à leitura final da página de dados correspondente.]*

3.4 Árvore B* (B*-Tree)

3.4.1 Princípio de Funcionamento e Implementação

A **Árvore B*** é uma evolução sofisticada da Árvore B que introduz duas premissas estruturais cruciais:

1. **Segregação entre Nós Internos e Nós Folhas:** Nós internos armazenam unicamente chaves delimitadoras de roteamento e apontadores para nós inferiores, não guardando dados de apontamento para páginas do arquivo de dados. Os dados reais de mapeamento (`key` e `pageIndex`) residem exclusivamente nos nós folhas (*Data Nodes*);

2. **Encadeamento Sequencial de Folhas:** As folhas contêm apontadores diretos para suas folhas irmãs subsequentes (`nextData`), permitindo varreduras sequenciais e consultas por intervalo (*range queries*) de altíssimo desempenho sem a necessidade de reescalar a árvore;

3. **Maior Taxa de Ocupação:** Mecanismos de redistribuição de chaves entre nós irmãos antes do particionamento garantem uma taxa média de ocupação superior ($\approx 66\%$ a 75%, contra os $\approx 50\%$ mínimos da Árvore B convencional), diminuindo ainda mais o número total de nós alocados.

No escopo deste projeto, a Árvore B* foi **especificada e implementada em memória principal** através da classe `BStarTreeMemory`. Diferentemente da Árvore B (que persiste seus nós em arquivo binário em disco), a Árvore B* mantém todos os seus nós de índice e dados alocados diretamente na memória primária (RAM), atuando como uma estrutura de indexação acelerada em memória para referenciar os blocos de dados persistidos no arquivo binário secundário.

Arquitetura e Implementação em Memória Principal: Implementada no *namespace* `Algorithm::BStarTree`:

- `IBStarTreeData::Node`: Estrutura híbrida baseada em união e discriminador de tipo (`Node::Type::Index` vs. `Node::Type::Data`):
  - O subtipo `Index` contém um vetor de chaves divisórias (`std::array<int, PAGE_SIZE> indexes`) e apontadores de subárvores (`nodePos`);
  - O subtipo `Data` contém as entradas de apontamento (`std::array<Entry, PAGE_SIZE> entries`) e o apontador sequencial para o próximo bloco de dados (`nextData`);
- `BStarTreeMemory`: Implementação da Árvore B* em **memória principal**. Gerencia os nós em memória através de um vetor dinâmico contíguo na RAM (arranjo `nodes_` de estruturas `Node`). Os apontadores de subárvores (`nodePos`) e o encadeamento sequencial entre folhas (`nextData`) são índices ordinais diretos dentro desse vetor na memória RAM, eliminando qualquer necessidade de I/O em disco para travessia entre nós;
- `BStarTree`: Fachada de orquestração que realiza a pesquisa pela chave na Árvore B* residente em RAM e, uma vez localizado o `pageIndex`, executa a única leitura necessária em memória secundária para recuperar o bloco no arquivo de dados.

Fase de Pré-processamento em Memória Principal: Durante o pré-processamento da Árvore B*:

1. O arquivo de dados original (em memória secundária) é percorrido sequencialmente em blocos de PAGE_SIZE registros (N/PAGE_SIZE leituras de disco);
2. À medida que cada página é carregada em memória primária, seus registros são inseridos na árvore B* residente em RAM via `InsertItem`;
3. Todas as rotinas de divisão de nós folhas (`SplitDataNode`) e de nós internos (`SplitIndexNode`), bem como a eventual criação de uma nova raiz em memória primária, são realizadas com velocidade de CPU na RAM, sem emitir operações de escrita (*write I/O*) em disco;
4. Por não gerar arquivo de cache persistente em disco, a fase a frio foca na leitura da entrada e montagem na memória volátil, enquanto no regime Warm o índice já se encontra instanciado na memória principal.

Fase de Pesquisa em Memória Principal: Ao consultar uma chave K:

1. A navegação inicia-se na raiz em memória RAM e transita rapidamente pelos nós internos (`Type::Index`) no vetor `nodes_`, comparando K com os divisores do nó para selecionar a subárvore filha sem nenhuma leitura de disco;

2. A busca descende na memória primária até alcançar obrigatoriamente um nó folha (`Type::Data`);

3. No nó folha residente em RAM, localiza-se a entrada associada à chave procurada;

4. Se a chave for encontrada, o `pageIndex` correspondente é resgatado e o algoritmo emite **exatamente uma única leitura de disco** na memória secundária via `File::GetPageAt(pageIndex)`, trazendo a página de dados com 100 itens para a RAM e recuperando o `Item` integral;

5. Se a chave não existir na folha, a pesquisa encerra imediatamente com **zero leituras de disco**;

6. Essa estratégia garante que o custo de I/O da pesquisa com sucesso seja fixo em $R = 1$ leitura de bloco, independentemente do volume N.

3.4.2 Resultados Experimentais e Tabela de Benchmark

A Tabela 5 expõe os resultados empíricos coletados para a Árvore B sob os regimes *Cold* e *Warm*. Os tempos de execução reportam os percentis p50, p75 e p99 obtidos diretamente a partir das 100 execuções sob as mesmas condições (com a chave garantidamente presente na entrada), além das medianas consolidadas de transferências e comparações (p50).

Tabela 5: Benchmark experimental – Árvore B*.

			Pré-processamento					Pesquisa				
			Tempo (ms)			Leituras	Comparações	Tempo (ms)			Leituras	Comparações
Situação	**Regime**	**Qtd (N)**	**p50**	**p75**	**p99**	**(p50)**	**(p50)**	**p50**	**p75**	**p99**	**(p50)**	**(p50)**
Ascendente	Cold	20	0.18	0.20	0.31	1	108	0.03	0.03	0.05	1	18
Ascendente	Cold	2.000	1.14	1.18	1.79	20	30.446	0.05	0.05	0.08	1	84
Ascendente	Cold	20.000	14.82	15.28	19.42	200	391.851	0.08	0.08	0.10	1	86
Ascendente	Cold	200.000	154.60	155.71	215.03	2.000	4.626.082	0.09	0.09	0.10	1	88
Ascendente	Cold	2.000.000	34094.70	34103.57	35398.01	20.000	52.971.485	0.10	1.03	1.34	1	90
Ascendente	Warm	20	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Ascendente	Warm	2.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Ascendente	Warm	20.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Ascendente	Warm	200.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Ascendente	Warm	2.000.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Descendente	Cold	20	0.18	0.19	0.24	1	176	0.03	0.03	0.05	1	21
Descendente	Cold	2.000	1.15	1.18	1.64	20	39.245	0.05	0.05	0.07	1	83
Descendente	Cold	20.000	14.39	14.79	15.70	200	472.478	0.08	0.08	0.09	1	84
Descendente	Cold	200.000	151.66	152.41	180.47	2.000	5.489.066	0.09	0.09	0.10	1	85
Descendente	Cold	2.000.000	36181.15	36191.88	36497.05	20.000	63.074.761	0.85	0.97	7.11	1	94
Descendente	Warm	20	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Descendente	Warm	2.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Descendente	Warm	20.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Descendente	Warm	200.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Descendente	Warm	2.000.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Desordenado	Cold	20	0.19	0.20	0.29	1	154	0.03	0.03	0.05	1	25
Desordenado	Cold	2.000	1.45	1.48	2.00	20	35.923	0.06	0.06	0.08	1	85
Desordenado	Cold	20.000	16.80	17.03	21.80	200	440.321	0.08	0.08	0.09	1	92
Desordenado	Cold	200.000	195.84	197.53	237.13	2.000	5.130.646	0.09	0.09	0.11	1	96
Desordenado	Cold	2.000.000	2059.14	2085.51	2349.36	20.000	57.387.954	0.09	0.10	0.11	1	108
Desordenado	Warm	20	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Desordenado	Warm	2.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Desordenado	Warm	20.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Desordenado	Warm	200.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0
Desordenado	Warm	2.000.000	0.00	0.00	0.00	0	0	0.00	0.00	0.00	0	0

Discussão dos Resultados: *[Espaço para análise do comportamento observado: comparar o comportamento da Árvore B* em memória principal em relação à Árvore B em memória secundária. Deve-se ressaltar que, por ter sido implementada em memória principal, a Árvore B* requer rigorosamente 1 leitura de disco por pesquisa com sucesso (exclusivamente para resgatar a página do arquivo de dados) e zero leituras caso a chave não esteja presente, eliminando totalmente o tráfego de I/O em disco associado à navegação no índice. Além disso, destacar a economia estrutural pela segregação de nós de índice e a eficiência em consultas sequenciais contíguas via encadeamento das folhas.]*

4 Comparação Geral e Discussão dos Algoritmos

A análise comparativa entre as quatro estruturas de dados permite identificar com clareza os *trade-offs* inerentes ao projeto de mecanismos de indexação para memória secundária.

4.1 Custo e Desempenho no Pré-processamento

A construção do índice representa um investimento inicial de tempo e operações de I/O em disco:

- **Acesso Sequencial Indexado:** Apresenta o menor custo de construção absoluto, efetuando uma única leitura linear sequencial no arquivo de dados ($O(N/\text{PAGE_SIZE})$ leituras). Contudo, pressupõe que os dados de entrada já estejam ordenados;

- **Árvore Binária de Pesquisa:** O custo de construção é altamente dependente da ordenação prévia. Em arquivos ordenados, a degeneração em lista produz $O(N^2)$ comparações e I/Os ao disco durante a inserção, tornando o processo inviável para grandes volumes de dados. Em contrapartida, com dados aleatórios, o custo converge para $O(N \log N)$;
- **Árvores B e B*:** Ambas exibem complexidade assintótica estável $O(N \log_M N)$ no pré-processamento independentemente da ordenação da entrada, porém com comportamentos de I/O diametralmente distintos decorrentes de sua residência de memória:
  - A **Árvore B (memória secundária)** realiza operações contínuas de gravação física e atualização de nós em disco no arquivo `.btree` durante o particionamento (*split*), incorrendo em custo perceptível de escrita em memória secundária no regime Cold;
  - A **Árvore B* (memória principal)** efetua as inserções e reestruturações de nós estritamente em memória RAM em tempo de CPU, limitando o consumo de I/O de disco às leituras sequenciais das páginas do arquivo de dados e eliminando integralmente as gravações de blocos de índice em disco, o que confere uma montagem em memória notavelmente mais rápida.

4.2 Eficiência na Fase de Pesquisa

Na pesquisa pontual por chave:

- **Número de Leituras de Disco:** O Acesso Sequencial Indexado, a Árvore B e a Árvore B* demonstram superioridade incontestável sobre a Árvore Binária. Enquanto a Árvore Binária pode requerer até N leituras de nós em disco no pior caso degenerado, a Árvore B e a Árvore B* ilustram o impacto da alocação de memória:
  - A **Árvore B (memória secundária)** lê cada nó percorrido da memória secundária para a principal ($h \approx \log_M N$ leituras de nós no arquivo `.btree`) e finaliza com 1 leitura da página de dados, totalizando entre 2 e 4 leituras de disco para até 2.000.000 de registros;
  - A **Árvore B* (memória principal)** percorre seus nós internos e folha inteiramente em memória RAM sem nenhuma leitura de disco no índice, necessitando de estritamente **1 única leitura de disco** (a transferência da página de dados onde o item reside via `File::GetPageAt`) em caso de chave presente, ou zero leituras em caso de chave ausente.
- **Comparações de Chaves:** A Árvore B e a Árvore B* realizam mais comparações em memória primária por nó (pesquisa dentro do arranjo de tamanho PAGE_SIZE), porém essa operação em RAM é dezenas de milhares de vezes mais veloz que uma única leitura adicional de disco, justificando plenamente a troca de processamento em CPU por economia de I/O. Na Árvore B*, como toda a árvore reside em RAM, essa vantagem temporal é ainda mais acentuada.

4.3 Sensibilidade ao Ordenamento Inicial da Entrada

- **Arquivos Ordenados (Ascendentes):** O ISA é a estrutura ideal pela simplicidade e compacidade; as Árvores B e B* comportam-se de forma estável e balanceada; a Árvore Binária degenera completamente;
- **Arquivos Descendentes:** O ISA torna-se inoperante; a Árvore Binária degenera de modo análogo ao caso ascendente; as Árvores B e B* mantêm invariantes estruturais e complexidade balanceada;
- **Arquivos Desordenados (Aleatórios):** O ISA não pode ser utilizado diretamente sem uma fase prévia de ordenação externa; a Árvore Binária atinge seu melhor caso médio ($O(\log N)$); as Árvores B e B* exibem sua máxima utilidade prática.

4.4 Impacto do Subsistema de Memória e Caches (Cold vs. Warm)

O contraste entre execuções *Cold* e *Warm* evidencia tanto o custo de persistência do índice quanto o papel amortecedor do subsistema de memória e do *Page Cache*:

- Em regime **Cold**, a inexistência prévia do arquivo de índice/cache impõe a obrigatoriedade de sua construção a partir do zero para as estruturas em memória secundária (ISA com `.cache_01`, Árvore Binária com `.binarytree` e Árvore B com `.btree`). Esse processo onera a fase de pré-processamento com a varredura integral da entrada e a emissão de sucessivas gravações físicas em disco para serializar a estrutura de dados, além de submeter a busca inicial às latências brutas do dispositivo de armazenamento. Na Árvore B*, o regime Cold corresponde à construção dinâmica da árvore em RAM a partir da varredura do arquivo de dados;
- Em regime **Warm**, o arquivo de índice já existe previamente no disco e é detectado pelo algoritmo (para ISA, Árvore Binária e Árvore B), ou a árvore já se encontra instanciada na memória RAM (para a Árvore B*). O pré-processamento resume-se à validação de integridade e carregamento dos blocos iniciais, usufruindo adicionalmente da retenção de páginas no *page cache* do sistema operacional na memória RAM, o que reduz os tempos de resposta em ordens de magnitude.

4.5 Síntese Comparativa Geral

A Tabela 6 consolida as principais características operacionais e assintóticas dos quatro métodos investigados.

Tabela 6: Quadro comparativo consolidado dos algoritmos de pesquisa externa.

Característica / Métrica	**ISA**	**Árvore Binária**	**Árvore B**	**Árvore B***
Residência do Índice	Memória Secundária	Memória Secundária	Memória Secundária	Memória Principal (RAM)
Complexidade de Busca (Médio)	$O(\log(N/B))$	$O(\log N)$	$O(\log_M N)$	$O(\log_M N)$
Complexidade de Busca (Pior Caso)	$O(\log(N/B))$	$O(N)$ (degenerada)	$O(\log_M N)$	$O(\log_M N)$
Acessos a Disco por Busca	$\approx \log_2(N/B) + 1$	$O(\log N)$ ou $O(N)$	$O(\log_M N) + 1$	1 (apenas pág. de dados)
Suporte a Dados Desordenados	Não	Sim	Sim	Sim
Sensibilidade à Ordem da Entrada	Extrema (requer asc.)	Extrema (degenera)	Mínima (imune)	Mínima (imune)
Espaço de Índice em Disco	Mínimo ($\approx 1/B$)	Moderado ($O(N)$)	Moderado	Zero (residente em RAM)
Fator de Ocupação Mínimo	100%	N/A	50%	$\approx$ 66% a 75%
Eficiência em Consultas por Faixa	Boa	Baixa	Moderada	Excelente (folhas encadeadas)

5 Conclusão

Este trabalho prático permitiu avaliar, quantitativa e qualitativamente, os princípios teóricos e os desafios práticos de engenharia associados à pesquisa em memória secundária.

A partir das análises experimentais consolidadas ao longo deste relatório, destacam-se as seguintes conclusões fundamentais:

1. **Custo Crítico de I/O:** Em sistemas de armazenamento persistente, o número de acessos à memória secundária (transferências de blocos/páginas) é a métrica preponderante que dita o tempo de execução total, sobrepujando com folga o custo de comparações de chaves efetuadas em memória RAM;

2. **Inadequação de Árvores Binárias Não Balanceadas:** A Árvore Binária Externa demonstrou-se inviável para uso geral em arquivos de grande porte, haja vista sua vulnerabilidade crítica à ordem dos dados, degenerando em listas lineares de profundidade inaceitável no pior caso;

3. **Elegância do Acesso Sequencial Indexado:** Quando os dados originais são estáticos e previamente ordenados de forma ascendente, o ISA oferece uma solução imbatível em simplicidade e consumo diminuto de espaço de índice, requerendo baixíssimo esforço de construção;

4. **Supremacia das Estruturas Multi-vias (Árvores B e B*):** Para sistemas modernos de gerenciamento de bancos de dados (SGBDs) e sistemas de arquivos, as árvores multi-vias consolidam-se como o padrão ouro da indústria devido ao elevado fator de ramificação que preserva profundidades ínfimas ($h \leq 3$) mesmo diante de milhões de registros. A bancada experimental permitiu contrastar diretamente dois paradigmas de alocação de memória:

   - A **Árvore B em memória secundária** validou a eficácia dos índices externos clássicos persistidos em disco (`.btree`), garantindo contenção estrita de acessos físicos a disco ($\approx \log_M N + 1$ blocos lidos) sem depender da disponibilidade de RAM para reter o índice integralmente;
   - A **Árvore B* em memória principal** demonstrou o potencial de arquiteturas aceleradas em memória primária, onde a navegação ocorre 100% em RAM com segregação de nós de roteamento, minimizando o custo de I/O em disco ao limite ótimo de exatamente 1 leitura de bloco de dados por consulta com sucesso.

Em suma, a implementação e instrumentação dos algoritmos na disciplina BCC203 forneceram sólida compreensão empírica das compensações de desempenho em sistemas de arquivos e bancos de dados, alinhando rigor formal algorítmico e práticas avançadas de desenvolvimento em C++ moderno.